指令概述

指令	作用	数据来源	目标	描述
OP_GETTABLE	表索引（动态键）	R[B]（table），R[C]（key）	R[A]	从寄存器 `R[B]` 指向的表中，以**运行期寄存器值** `R[C]` 作为 key 取值，结果写入 `R[A]`。等价于：`R[A] = R[B][R[C]]`
OP_GETI	表索引（整数键）	R[B]（table），C（立即数整数）	R[A]	从表 `R[B]` 中以**立即数整数 C** 作为索引取值，结果写入 `R[A]`。用于优化数组访问，避免构造 key 对象。等价于：`R[A] = R[B][C]`
OP_GETFIELD	表字段访问（短字符串键）	R[B]（table），K[C]（短字符串常量）	R[A]	从表 `R[B]` 中以**常量池中的短字符串** `K[C]` 作为 key 取值，结果写入 `R[A]`。常用于 `obj.field` 形式。等价于：`R[A] = R[B][K[C]]`

当然也推荐各位去看一下指令集注释，写的非常清晰清楚了

OP_GETI

功能简述

这个的感觉像是你 C 代码中的索引 `arrr[i]`，立即访问无须偏移，我们来看代码实现

代码实现

```
// A B C   R[A] := R[B][C]
L_OP_GETI: {
    StkId ra = RA(i);
    TValue *rb = vRB(i);
    int c = GETARG_C(i);
    lu_byte tag;
    luaV_fastgeti(rb, c, s2v(ra), tag);
    if (tagisempty(tag)) {
        TValue key;
        setivalue(&key, c);
        Protect(luaV_finishget(L, rb, &key, ra, tag));
    }
    vmbreak;
}
```

先看我写上去的指令集注释，寄存器内获取到 B，B 在 Lua 中是一个类似数组的类型，使用 C 来进行索引，最后给 A

那么 A 的值就是目标写入，B[C]获取的就是源操作数了，往下看具体代码

```
// 整数“luaV_fastget”的特殊情况，内联快速情况 'luaH_getint'。
#define luaV_fastgeti(t, k, res, tag)                    \
    if (!ttistable(t))                              \ // 判断是否属于表类型
        tag = LUA_VNOTABLE;                     \
    else {                                          \
        luaH_fastgeti(hvalue(t), k, res, tag); \
    }

#define luaH_fastgeti(t, k, res, tag)                       \
    {                                                   \
        Table *h = t;                               \
        lua_Unsigned u = l_castS2U(k) - 1u;         \ // Lua特性，下标从1开始，
l_castS2U(k) 把 k 转为无符号整型，避免负数或溢出
        if ((u < h->asize)) {                       \ // 判断索引是否落在数组部分
[0, asize-1] 内
            tag = *getArrTag(h, u);             \ // 这个宏的意思是返回数组第 k 个
元素的tag存储位置 一个 lu_byte* 跳过数组头部
            if (!tagisempty(tag)) {             \ // 判断一下上面有没有拿到，如果拿
到了就开始赋值操作
                farr2val(h, u, tag, res); \ // 这里就是很直白的赋值了
            }                                   \
        } else {                                    \
```

```
            tag = luaH_getint(h, (k), res);   \ 否则去哈希表部分找整数索引
        }                                          \
    }
lu_byte luaH_getint(Table *t, lua_Integer key, TValue *res)
{
    unsigned k = ikeyinarray(t, key); // 判断 key 是否落在数组部分
    if (k > 0) {
        lu_byte tag = *getArrTag(t, k - 1); // 直接取数组第 k-1 个元素的 tag，然后同上逻辑
        if (!tagisempty(tag))
            farr2val(t, k - 1, tag, res);
        return tag;
    } else
        return finishnodeget(getintfromhash(t, key), res); //
}
static const TValue *getintfromhash(Table *t, lua_Integer key)
{
    Node *n = hashint(t, key); // 找到 key 对应 hash bucket
    lua_assert(!ikeyinarray(t, key)); // 断言 key 不在数组部分
    for (;;) { /* check whether 'key' is somewhere in the chain 遍历链表 */
        if (keyisinteger(n) && keyival(n) == key)
            return gval(n); /* that's it  找到整数 key，返回 value 指针 */
        else {
            int nx = gnext(n); // 指向下一个 node 的偏移
            if (nx == 0)
                break; // 链表到尾，没找到
            n += nx; // 继续遍历
        }
    }
    return &absentkey; // 没找到 key，返回全局 absentkey
}
static lu_byte finishnodeget(const TValue *val, TValue *res)
{
    if (!ttisnil(val)) {
        setobj(((lua_State *)NULL), res, val); // 拷贝 value 到 res
    }
    return ttypetag(val); // 返回类型 tag
}
```

一切尽在注释中，我让 AI 搞了一个简单的图表

```
Lua VM 执行 GETI(A,B,C):
  R[A] = R[B][C]
```

```
内部调用:
  luaV_fastgeti(t, k, res, tag)
    └-> luaH_fastgeti(hvalue(t), k, res, tag)
          ├- 数组部分 fast path:
          │     tag = *getArrTag(h, u)
          │     farr2val(h,u,tag,res)
          └- 哈希表 slow path:
                tag = luaH_getint(h, k, res)
                        ├- 数组部分检测 ikeyinarray → false
                        ├- getintfromhash → 找哈希 node
                        └- finishnodeget → 写 res，返回 tag
```

luaV_finishget 这个推荐自己分析一下，我懒得写了，如果你弄不懂就直接找我，简单总结一下就是这几点：

- `luaV_finishget` = **Lua 的 __index 元表链访问**
- 先判断目标是不是 table
- 再找元方法 → 如果是函数直接调用 → 如果是 table 继续访问
- 有循环限制，防止元方法链太长导致无限循环
- **最终 fallback** → nil

OP_GETTABLE

功能概述

OP_GETTABLE 的感觉就像二维数组，第二维度内只有两个元素，key 和 value

代码实现

```
L_OP_GETTABLE: {
    StkId ra = RA(i);
    TValue *rb = vRB(i);
    TValue *rc = vRC(i);
    lu_byte tag;
```

```
    if (ttisinteger(rc)) { /* 是否属于整数类型 */
        luaV_fastgeti(rb, ivalue(rc), s2v(ra), tag);
    } else
        luaV_fastget(rb, rc, s2v(ra), luaH_get, tag);
    if (tagisempty(tag))
        Protect(luaV_finishget(L, rb, rc, ra, tag));
    vmbreak;
}
```

A 还是一个目标寄存器，B 是 Lua 数据结构中的表，C 是 key，通过 C 再 B 中获取值，我们来看详细代码

```
#define cast_u(o) cast(union GCUnion *, (o))
#define gco2t(o) check_exp((o)->tt == LUA_VTABLE, &((cast_u(o))->h))
#define hvalue(o) check_exp(ttistable(o), gco2t(val_(o).gc))
#define luaV_fastget(t, k, res, f, tag) (tag = (!ttistable(t) ? LUA_VNOTABLE
: f(hvalue(t), k, res)))
```

先判断 **R[B] 的类型是不是 table**，如果不是表就直接返回 `LUA_VNOTABLE`，告诉 VM 这个值不是 table；如果是表，那就：

1. **取出 Table 指针**（`hvalue(R[B])`）
2. **用 key R[C] 查数组/哈希表**
   - **快速路径**：如果 key 对应数组 slot，有值就直接填入 R[A]
   - **慢路径**：如果 key 不在数组，查哈希表节点
3. **检查是否找到值**
   - 找到 → 把值写入 R[A]，返回类型 tag
   - 没找到 → 调用 `luaV_finishget` 处理元方法
4. **元表 fallback**（__index）
   - 如果 table/metatable 有 `__index`，继续访问元表或调用函数
   - 元方法链找不到 → R[A] = nil，tag = LUA_TNIL

OP_GETFIELD

功能概述

OP_GETFIELD 更为复杂，实际情况可能是你拿一个变量名内的数据当作 key 去取值，需要从 K 中先找到 C，然后再去从 B 中取真实需要的值

代码实现

```
L_OP_GETFIELD: {
    StkId ra = RA(i);
    TValue *rb = vRB(i);
    TValue *rc = KC(i);
    TString *key = tsvalue(rc); /* key must be a short string */
    lu_byte tag;
    luaV_fastget(rb, key, s2v(ra), luaH_getshortstr, tag);
    if (tagisempty(tag))
        Protect(luaV_finishget(L, rb, rc, ra, tag));
    vmbreak;
}
```

这个当作练习吧，毕竟前面都有了，各位可以整理一下你的理解然后自己做一下笔记